

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ACADEMIC ASSIGNMENT

--

CO4 - AT 1
CODING CHALLENGE

Network Topology Design: Coding Challenge

Name	S. Lokesh
Reg. No	192572151
Course Name	Computer Networks
Course Code	CSA0707
Assignment	AT 1 - Coding Challenge
Date of Submission	27 August 2026

Declaration

I hereby declare that this assignment is my own original work, prepared under the guidance of my course faculty, and has been submitted in accordance with the academic requirements of the course.

Question 1: Python Program to Generate a Star Topology

A star topology connects every host to a single central switch. The program below accepts the number of hosts, reads the Cat6 UTP cable length for each host, displays each connection in an ASCII-style form, and finally verifies whether all hosts are correctly connected through the central switch. A maximum cable length of 100 metres is considered for each connection.

Python Program

```
# Star Topology Generator
n = int(input("Enter number of hosts: "))
switch = "Central_Switch"
connections = []

for i in range(1, n + 1):
    host = "Host" + str(i)
    length = float(input("Enter cable length for " + host + " (m): "))
    if 0 < length <= 100:
        connections.append(host)
        print(host, "---- Cat6", length, "m ----", switch)
    else:
        print(host, ": Invalid cable length")

print("\nTopology Verification")
if len(connections) == n:
    print("All hosts are connected to the central switch.")
else:
    print("Some hosts are not properly connected.")
```

Sample Output

```
Enter number of hosts: 3
Host1 ---- Cat6 20 m ---- Central_Switch
Host2 ---- Cat6 30 m ---- Central_Switch
Host3 ---- Cat6 25 m ---- Central_Switch
All hosts are connected to the central switch.
```

Explanation

In a star network, every host has an independent cable connection to the central switch, and communication between hosts passes through that switch. If one host cable fails, only that host is affected; the other hosts continue to communicate normally.

Question 2: Function to Validate an Ethernet UTP Cable Segment

The function `validate_segment(length_m)` checks whether a UTP cable segment satisfies the given Ethernet limit of 100 metres. The function returns a success status for valid values and an error message for invalid or excessive cable lengths. Three test cases are used to demonstrate the correctness of the program.

Python Program

```
def validate_segment(length_m):
    if length_m <= 0:
        return False, "Error: Length must be greater than 0 m."
    elif length_m <= 100:
        return True, "Valid Ethernet cable segment."
    else:
        return False, "Error: Cable length exceeds 100 m."

test_cases = [50, 100, 120]

for length in test_cases:
    status, message = validate_segment(length)
    print("Cable Length:", length, "m")
    print("Status:", status)
    print("Message:", message)
    print()
```

Output

```
Cable Length: 50 m  Status: True
Message: Valid Ethernet cable segment.
Cable Length: 100 m  Status: True
Message: Valid Ethernet cable segment.
Cable Length: 120 m  Status: False
Message: Error: Cable length exceeds 100 m.
```

Interpretation

The 50 m test is accepted because it is below the maximum allowed length. The 100 m test is also accepted because it is exactly at the permitted limit. The 120 m test is rejected because it exceeds the 100 m limit. The extra validation for zero or negative values prevents meaningless cable lengths from being accepted. Excessive copper cable length can increase attenuation and may reduce the quality

Question 3: Simulation of CSMA/CD with Four Hosts

CSMA/CD is a media-access method used in traditional shared Ethernet. The following simulation uses four hosts. Each host makes a transmission attempt; when a collision is generated, the program records it, chooses a random back-off interval, waits, and tries again. Every attempt, collision and back-off value is written into csma_log.txt.

Python Program

```
import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]
log = open("csma_log.txt", "w")

for host in hosts:
    attempt = 1
    success = False
    while not success:
        log.write(host + " Attempt " + str(attempt) + "\n")
        collision = random.choice([True, False])
        if collision:
            backoff = random.randint(1, 5)
            print(host, "- Collision detected")
            print("Back-off:", backoff, "seconds")
            log.write("Collision detected\n")
            log.write("Back-off: " + str(backoff) + " seconds\n")
            time.sleep(1)
            attempt += 1
        else:
            print(host, "- Frame transmitted successfully")
            log.write("Transmission successful\n\n")
            success = True

log.close()
print("Events saved in csma_log.txt")
```

Sample Output

```
Host1 - Collision detected | Back-off: 3 seconds
Host1 - Frame transmitted successfully
```

Host2 - Frame transmitted successfully

Host3 - Collision detected | Back-off: 2 seconds

Host3 - Frame transmitted successfully

Analysis

Before transmitting, a station must determine whether the medium is available. If two stations attempt to transmit at the same time, a collision can occur. CSMA/CD responds by stopping the current transmission and making each station wait for a randomly selected back-off period before trying again. Random waiting reduces the probability that the same hosts will collide repeatedly. The log file is useful for studying the number of attempts, collision events, and waiting periods. Thus, the simulation demonstrates how collision detection and randomized retransmission can reduce repeated transmission conflicts and improve the effective use of a shared Ethernet medium.

Question 4: Client-Server Communication and MAC Address Learning

This client-server application uses Python sockets to represent communication between hosts through a central switch. The client sends a simulated Ethernet frame containing its source MAC address, port number, and message. The server learns the source MAC address, stores the corresponding port in a MAC address table, displays the updated table, and sends an acknowledgement to the client.

Server Program

```
import socket

server = socket.socket()
server.bind(("localhost", 5000))
server.listen(1)
print("Switch Server is waiting...")
mac_table = {}

conn, address = server.accept()
data = conn.recv(1024).decode()
mac, port, message = data.split(",")
mac_table[mac] = port

print("Frame Received:", message)
print("MAC Address Table")
for m, p in mac_table.items():
    print(m, "->", p)

conn.send("Frame received successfully".encode())
conn.close()
server.close()
```

Client Program

```
import socket

client = socket.socket()
client.connect(("localhost", 5000))
frame = "AA:BB:CC:DD:EE:01,Port1,Hello Host"
client.send(frame.encode())
print(client.recv(1024).decode())
client.close()
```

Sample Result

Frame Received: Hello Host

MAC Address Table

AA:BB:CC:DD:EE:01 -> Port1

Client: Frame received successfully

Explanation

When a switch receives an Ethernet frame, it examines the source MAC address and associates that address with the incoming port. This process is called MAC learning. The learned information is stored in the MAC address table. When a destination MAC address is known, the switch can forward the frame only through the required port instead of sending it everywhere. If the destination is not yet known, the switch may flood the frame through other ports until the destination is learned. MAC learning therefore reduces unnecessary traffic and improves Ethernet efficiency. This socket example demonstrates frame reception, MAC address learning, table maintenance and basic forwarding behaviour in a star-topology network.

Question 5: StarSwitch Class for Port Management and Broadcasting

The StarSwitch class models basic Ethernet-switch behaviour. It keeps a record of currently connected hosts and provides methods to add a host, remove a host and broadcast a frame. During broadcasting, the source host is excluded and the frame is delivered to all other active hosts. Each broadcast operation is also recorded in a log file.

Python Program

```
class StarSwitch:
    def __init__(self):
        self.ports = []

    def add_port(self, host_id):
        if host_id not in self.ports:
            self.ports.append(host_id)
            print(host_id, "connected")

    def remove_port(self, host_id):
        if host_id in self.ports:
            self.ports.remove(host_id)
            print(host_id, "disconnected")

    def broadcast(self, source, frame):
        receivers = []
        print("\nSource:", source, " Frame:", frame)
        for host in self.ports:
            if host != source:
                receivers.append(host)
                print("Received by:", host)
        with open("broadcast_log.txt", "a") as log:
            log.write(source + " -> " + str(receivers) + "\n")

switch = StarSwitch()
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")
switch.broadcast("Host1", "Hello Network")
```



```
switch.remove_port("Host4")  
switch.broadcast("Host2", "New Frame")
```

Sample Output

```
Host1 connected  Host2 connected  Host3 connected  Host4 connected  
Source: Host1  Frame: Hello Network  
Received by: Host2, Host3, Host4  
Host4 disconnected  
Source: Host2  Frame: New Frame  
Received by: Host1, Host3
```

Explanation

The `add_port()` method registers a new host in the active-port list, while `remove_port()` disconnects an existing host. The `broadcast()` method sends a frame to every connected host except the source and stores the source and receiver list in `broadcast_log.txt`. Broadcasting is important when a frame must reach all devices in a broadcast domain or when the switch does not yet know the exact destination port. The class also shows that port membership changes dynamically when devices are connected or disconnected. Therefore, the program demonstrates basic switch port management, broadcast frame delivery, active-host tracking, and logging of Ethernet communication in a star network.